

Diario di lavoro

1. Obiettivo di questa fase

Estendere il testbed OpenWrt della tesi precedente (Raspberry Pi 4 + adattatore USB MediaTek MT7612U) per supportare OpenAppFilter (OAF), uno strumento open source per il filtraggio del traffico a livello applicativo su OpenWrt, come base per lo Stadio 1 (riconoscimento rapido del contesto del traffico) della nuova proposta di tesi sul traffic fingerprinting.

OAF richiede una corrispondenza esatta tra il modulo kernel precompilato e la versione OpenWrt in uso (controllo "magic number" sul kernel). La versione 23.05.4 usata finora non ha una release OAF disponibile: è stato quindi necessario aggiornare il testbed a una versione più recente.

2. Scelta della versione OpenWrt

Confronto tra le versioni disponibili e motivazione della scelta finale:

Versione OpenWrt	Package manager	Note
23.05.4	opkg	Versione di partenza (tesi precedente). Stabile, senza supporto OAF.
24.10.5 / 24.10.x	opkg	Prima versione con release OAF disponibile. Scartata: build OAF più datata.
25.12.0 / 25.12.2	apk (transizione)	OAF ha release compilate per queste versioni.
25.12.2	apk	VERSIONE SCELTA. Corrisponde al kernel 6.12.74 dichiarato nel pacchetto OAF v6.1.8 per target bcm27xx_bcm2711.
25.12.4	apk	Versione più recente disponibile, supportata anche da OAF, ma non scelta per allineamento kernel più stretto con la 25.12.2.

Motivazione della scelta (25.12.2 invece di 25.12.4): il nome del pacchetto OAF più recente (v6.1.8) per il target del Raspberry Pi 4 riporta esplicitamente "openwrt25.12.2-oaf_v6.1.8-bcm27xx_bcm2711-kernel_6.12.74", indicando un allineamento diretto con questa versione specifica. Si è preferito seguire questa indicazione puntuale piuttosto che presumere compatibilità con la 25.12.4.

Nota metodologica: a differenza della tesi precedente (dove si era scelto di evitare la transizione opkg→apk per instabilità dei repository), qui si accetta apk perché la 25.12.x è ormai una release stabile da diversi mesi, non più in fase di rodaggio come lo era la 24.x/25.x snapshot all'epoca dei test precedenti.

3. Fase 0 — Backup di sicurezza

Prima di modificare la configurazione esistente, è stato eseguito un backup completo della SD funzionante con OpenWrt 23.05.4, per avere un punto di ripristino in caso di problemi con la nuova versione.

- Strumento utilizzato: Win32 Disk Imager (Windows), funzione "Read"
- File generato: backup_openwrt_23.05.img
- Percorso: Desktop → "copia immagine raspberry openwrt v23"
- Nota: Raspberry Pi Imager (versione 2.0.6 in uso) non dispone nativamente di una funzione di backup/lettura della SD, è stato necessario uno strumento esterno.

Backup completato con successo.

4. Fase 1 — Generazione della nuova immagine OpenWrt 25.12.2

4.1 Ambiente di build

La generazione è avvenuta su una macchina virtuale Lubuntu (VirtualBox), usando l'Image Builder ufficiale OpenWrt, non la compilazione completa da sorgente (che sarebbe stata necessaria solo per OAF su versioni non coperte da release precompilate, opzione scartata per complessità/tempi, come già valutato e scartato in precedenza per il chipset Realtek RTL88x2BU).

4.2 Download e preparazione

```
wget https://downloads.openwrt.org/releases/25.12.2/targets/bcm27xx/bcm2711/openwrt-imagebuilder-25.12.2-bcm27xx-bcm2711.Linux-x86_64.tar.zst
```

Nota tecnica: a differenza della versione 23.05.4 (archivio .tar.xz), la 25.12.2 utilizza il formato di compressione .tar.zst, che ha richiesto l'installazione dell'utility zstd sulla VM (non presente di default):

```
sudo apt update
sudo apt install -y zstd

tar --zstd -xf openwrt-imagebuilder-25.12.2-bcm27xx-bcm2711.Linux-x86_64.tar.zst
cd openwrt-imagebuilder-25.12.2-bcm27xx-bcm2711.Linux-x86_64
```

4.3 Generazione dell'immagine

Comando di build, con lo stesso target hardware (Raspberry Pi 4, profilo rpi-4) e la stessa lista di pacchetti core già validata:

```
make image PROFILE="rpi-4" PACKAGES="kmod-usb-net-ipheth usbmuxd libimobiledevice kmod-mt76x2u usbutils iw iwinfo nano tcpdump"
```

La build ha installato con successo 185 pacchetti totali (i 9 richiesti esplicitamente più tutte le dipendenze di sistema: busybox, libc, libgcc, apk-mbedtls, zlib, ecc.), confermando che tutti i pacchetti necessari sono disponibili nel repository della versione 25.12.2.

4.4 Pacchetti inclusi esplicitamente e loro funzione

Pacchetto	Funzione
kmod-mt76x2u	Driver per il chipset MediaTek MT7612U (adattatore Wi-Fi USB). Permette di usarlo come radio wireless (AP/client).
kmod-usb-net-ipheth	Driver del protocollo Apple per il tethering USB. Fa comparire l'iPhone collegato via cavo come interfaccia di rete (eth1).
usbmuxd	Demone che gestisce la comunicazione USB con i dispositivi iOS, necessario per il tethering Apple.
libimobiledevice	Libreria che implementa il protocollo di comunicazione con i dispositivi iOS, usata da usbmuxd.
usbutils	Strumenti diagnostici USB (es. lsusb) per verificare i dispositivi collegati.
iw	Strumento a riga di comando per configurare/interrogare le interfacce wireless.
iwinfo	Libreria/tool OpenWrt per informazioni wireless, usato anche internamente da LuCI.
nano	Editor di testo da terminale per modificare i file di configurazione.
tcpdump	Strumento di cattura pacchetti per l'analisi del traffico di rete.

4.5 File generati

L'Image Builder produce quattro varianti di immagine; quella corretta per il primo flash su una SD (a differenza di "sysupgrade", pensato per aggiornare un sistema già attivo) è la versione "factory":

- openwrt-25.12.2-bcm27xx-bcm2711-rpi-4-ext4-factory.img.gz ← FILE DA USARE PER IL FLASH (20,8 MB)
- openwrt-25.12.2-bcm27xx-bcm2711-rpi-4-ext4-sysupgrade.img.gz
- openwrt-25.12.2-bcm27xx-bcm2711-rpi-4-squashfs-factory.img.gz
- openwrt-25.12.2-bcm27xx-bcm2711-rpi-4-squashfs-sysupgrade.img.gz

4.6 Trasferimento del file sul PC host

La VM Ubuntu non dispone di cartella condivisa configurata con l'host. Il trasferimento del file .img.gz è stato effettuato tramite WeTransfer.

Immagine generata e trasferita con successo sul PC host.

5. Fase 2 — Flash e primo boot

L'immagine è stata flashata sulla SD di test (distinta da quella di backup con OpenWrt 23.05.4) tramite Raspberry Pi Imager, opzione "Use custom", selezionando direttamente il file .img.gz generato al passo precedente.

Al primo boot, l'adattatore MediaTek risultava correttamente riconosciuto a livello USB (confermato con lsusb, che mostra l'ID MediaTek Inc. MT7612U) e il relativo modulo kernel risultava caricato (confermato con lsmod | grep mt76: catena completa mt76, mt76_usb, mt76x02_lib, mt76x02_usb, mt76x2_common, mt76x2u).

6. Fase 3 — Configurazione dell'Access Point e diagnosi instabilità

6.1 Configurazione iniziale dell'interfaccia wireless

La configurazione wireless è stata rigenerata da zero per includere correttamente la radio corrispondente all'adattatore MediaTek (riconosciuto dal sistema in base al proprio percorso fisico USB, non con un nome fisso):

```
rm /etc/config/wireless
wifi config
```

Il comando rigenera /etc/config/wireless rilevando automaticamente tutte le radio presenti nel sistema, ma su questa immagine 25.12.2 l'ordine di enumerazione si è rivelato non fisso: a seconda del momento del boot e della porta USB fisica occupata, il MediaTek può comparire come radio0, radio1 o radio2. La sezione corretta è stata identificata di volta in volta osservando il campo option path di ciascuna sezione: i percorsi contenenti la stringa "usb" identificano sempre l'adattatore USB esterno, quelli contenenti "mmcnr/mmc_host" identificano il chip Broadcom interno (mai utilizzato in questa configurazione).

Una volta identificata la sezione corretta (nell'esempio, radio0), è stata applicata la configurazione dell'Access Point tramite UCI:

```
uci set wireless.radio0.disabled='0'
uci set wireless.default_radio0.ssid='TestAP'
uci set wireless.default_radio0.encryption='psk2'
uci set wireless.default_radio0.key='pass1234'
uci set wireless.default_radio0.network='lan'
uci commit wireless
wifi reload
```

Errore iniziale riscontrato e corretto: UCI distingue due livelli di attivazione indipendenti per ciascuna radio, il dispositivo fisico (wireless.radioN.disabled) e la specifica interfaccia AP sovrapposta ad esso (wireless.default_radioN.disabled). Il primo tentativo di configurazione aveva abilitato solo il dispositivo radio, lasciando l'interfaccia default_radioN ancora disabled='1': la radio risultava accesa ma senza alcuna rete Wi-Fi effettivamente trasmessa. Impostando esplicitamente disabled='0' su entrambe le sezioni, l'interfaccia è passata correttamente a type AP, verificabile tramite iw dev:

```
phy#0
    Interface phy0-ap0
        ssid TestAP
        type AP
        channel 36 (5180 MHz), width: 80 MHz, center1: 5210 MHz
        txpower 20.00 dBm
```

6.2 Diagnosi e risoluzione dell'instabilità dell'Access Point

Dopo il primo successo nella configurazione, l'Access Point risultava instabile: a ogni riavvio del sistema (o in seguito a disconnessioni USB spontanee) l'interfaccia tornava in type managed, richiedendo di ripetere l'intera procedura di rigenerazione della configurazione. La diagnosi ha richiesto diversi passaggi, riassunti nella tabella seguente:

Aspetto	Dettaglio
Sintomo	L'interfaccia MediaTek passava da type AP a type managed pochi secondi dopo ogni wifi reload; nei log comparivano errori mt76x02u_mcu_wait_resp failed e ripetuti over-current change su tutte le porte USB.
Prima ipotesi testata	Cavo USB-iPhone di qualità migliore (con supporto fast-charging) come possibile causa dell'assorbimento eccessivo. Ripristinato il cavo precedente: il problema persisteva ugualmente → ipotesi scartata come causa principale.
Causa reale	Budget di corrente complessivo insufficiente sull'alimentazione USB del Raspberry Pi, condiviso tra adattatore MediaTek, tethering iPhone, tastiera USB e hub interno. Il sovraccarico causava la disconnessione fisica e la ri-enumerazione dell'adattatore su una porta USB diversa ad ogni caduta.
Effetto collaterale	Ogni volta che il MediaTek si ri-enumerava su una porta diversa, il relativo option path in /etc/config/wireless (associato alla vecchia porta) non corrispondeva più all'hardware reale: la sezione radioN restava configurata ma senza device fisico dietro, e hostapd restava attivo senza mai portare l'interfaccia in AP.
Risoluzione	Sostituzione del cavo di collegamento iPhone-Raspberry Pi; rigenerazione della configurazione wireless (rm /etc/config/wireless + wifi config) per rimappare correttamente radioN sul path USB attuale del MediaTek.

Nota metodologica: La configurazione wireless di OpenWrt lega esplicitamente ciascuna radio al proprio percorso fisico nel bus USB, non a un identificativo persistente del dispositivo. Ne consegue che, in presenza di instabilità elettrica, la configurazione può divergere silenziosamente dall'hardware reale, con hostapd regolarmente in esecuzione ma privo di un'interfaccia fisica valida da gestire.

Comando diagnostico risolutivo, utile per mappare in modo affidabile ciascuna interfaccia ieee80211 al proprio percorso fisico indipendentemente dalla numerazione radioN assegnata da UCI:

```
for p in /sys/class/ieee80211/*/device; do echo "$p -> $(readlink -f $p)"; done
```

Una volta identificata la corrispondenza corretta tra radioN e percorso USB attuale, la configurazione wireless è stata rigenerata (rm /etc/config/wireless + wifi config) e riapplicata con gli stessi comandi UCI, questa volta sulla sezione risultata effettivamente corrispondente al MediaTek.

Access Point TestAP attivo e stabile su adattatore MediaTek (interfaccia phy0-ap0, type AP, canale 36 / 5180 MHz, banda 80 MHz).

7. Fase 4 — Tethering iPhone e firewall

Riconfigurazione dell'interfaccia di tethering USB verso iPhone:

```
uci set network.iphone=interface
uci set network.iphone.proto='dhcp'
uci set network.iphone.device='eth1'
uci commit network
/etc/init.d/network restart
```

Verifica dell'assegnazione IP sull'interfaccia eth1 tramite `ip a show eth1`: inizialmente l'interfaccia risultava in stato down senza indirizzo IP assegnato. Dopo il comando sopra, l'interfaccia è risultata correttamente in stato up con indirizzo IP assegnato dall'iPhone in tethering.

Aggiunta dell'interfaccia iphone alla zona WAN del firewall, per abilitare il masquerading (NAT) del traffico dei client verso Internet:

```
uci add_list firewall.@zone[1].network='iphone'
uci commit firewall
/etc/init.d/network restart
/etc/init.d/firewall restart
```

8. Verifica di connettività end-to-end

Test eseguito con successo: un client connesso alla rete TestAP risulta in grado di raggiungere Internet (ping riuscito verso 8.8.8.8), confermando il corretto funzionamento dell'intera catena:

Access Point (MediaTek) → bridge LAN → NAT/masquerading (firewall) → tethering USB (eth1) → rete dati iPhone.

Connettività Internet end-to-end confermata dal client Wi-Fi.

9. Fase 5 — Script di auto-correzione del percorso USB al boot

Per evitare di dover ripetere manualmente, a ogni riavvio, la rimappatura della sezione radioN sul percorso USB attuale del MediaTek, è stato creato uno script che esegue automaticamente questa correzione.

```
#!/bin/sh
for phy in /sys/class/ieee80211/*; do
    path=$(readlink -f "$phy/device")
    case "$path" in
        *usb*)
            ucipath=$(echo "$path" | sed 's#^/sys/devices/platform/##')
            iface=$(uci show wireless | grep "ssid='TestAP'" | cut -d. -f2)
            dev=$(uci get wireless.$iface.device)
            uci set wireless.$dev.path="$ucipath"
            uci commit wireless
            ;;
    esac
done
exit 0
```

Lo script è stato inizialmente creato in `/etc/uci-defaults/` (percorso eseguito automaticamente una sola volta al primo boot dopo la generazione dell'immagine), poi spostato in una posizione stabile per poter essere richiamato ad ogni riavvio:

```
chmod +x /etc/uci-defaults/99-fix-mediatek-path
mv /etc/uci-defaults/99-fix-mediatek-path /etc/fix-mediatek-path.sh
```

Per garantirne l'esecuzione automatica ad ogni boot (non solo al primo), le seguenti righe sono state aggiunte a `/etc/rc.local`, subito prima della riga `exit 0`:

```
/etc/fix-mediatek-path.sh
wifi reload
```

Script di auto-correzione attivo: il percorso USB del MediaTek viene rimappato automaticamente ad ogni riavvio, senza intervento manuale.

10. Fase 6 — Verifica e sostituzione del pacchetto wpad

Verifica della versione di wpad effettivamente installata:

```
apk list --installed | grep wpad
```

Come atteso, risultava installato il pacchetto base wpad-basic-mbedtls, privo del supporto completo agli standard di gestione avanzata e roaming Wi-Fi (802.11k/v/r). Sostituzione con la build completa:

```
apk update
apk del wpad-basic-mbedtls
apk add wpad-openssl
```

11. Fase 7 — Installazione e configurazione di OpenAppFilter (OAF)

11.1 Download dei pacchetti OAF (v6.1.8)

Pacchetti scaricati direttamente sul Raspberry Pi via terminale, all'interno della cartella temporanea /tmp:

```
cd /tmp
wget -O luci-app-oaf-6.1.4-r1.apk
https://github.com/destan19/OpenAppFilter/releases/download/v6.1.8/luci-app-oaf-6.1.4-r1.apk
wget -O oaf-kernel.tar.gz
https://github.com/destan19/OpenAppFilter/releases/download/v6.1.8/openwrt25.12.2-oaf_v6.1.8-bcm27xx_bcm2711-kernel_6.12.74.tar.gz
```

11.2 Estrazione e installazione tramite apk

Estrazione dell'archivio del kernel e installazione in sequenza, forzando l'accettazione dei pacchetti non firmati ufficialmente (--allow-untrusted, necessario per pacchetti di terze parti non presenti nei repository OpenWrt ufficiali):

```
tar -zxvf oaf-kernel.tar.gz
cd openwrt25.12.2-oaf_v6.1.8-bcm27xx_bcm2711-kernel_6.12.74/
apk add --allow-untrusted kmod-oaf*.apk appfilter*.apk
cd /tmp
apk add --allow-untrusted luci-app-oaf-6.1.4-r1.apk
```

11.3 Problema riscontrato: caricamento del modulo kernel

Il comando standard depmod (necessario per indicizzare i moduli kernel disponibili) non è risultato disponibile nell'ambiente BusyBox minimale di questa immagine OpenWrt (errore: -ash: depmod: not found). Questo impediva anche l'esito del comando modprobe, che falliva con l'errore "failed to find a module named oaf", non riuscendo a localizzare il modulo appena installato.

Risoluzione: caricamento manuale forzato del modulo, bypassando l'indicizzazione automatica, direttamente dal percorso assoluto nel filesystem:

```
insmod /lib/modules/6.12.74/oaf.ko
```

Verifica dell'esito tramite i log del kernel, con conferma della corretta inizializzazione del driver:

```
dmesg | grep oaf
# output: oaf: Driver ver. 5.3.3 ... oaf: init ok
```

Verifica dello stato del servizio:

```
/etc/init.d/appfilter status  
# output: running
```

11.4 Persistenza del modulo al boot

Per evitare di dover ripetere insmod manualmente ad ogni riavvio, il modulo è stato registrato nella configurazione di sistema dei moduli da caricare automaticamente:

```
echo "oaf" > /etc/modules.d/oaf
```

OpenAppFilter installato, modulo kernel caricato correttamente e persistente al boot. Servizio appfilter attivo.

12. Fase 8 — Stadio 1: individuazione delle regole di riconoscimento (Google Play / App Store)

12.1 Struttura dei file di regole

OAF gestisce le firme di riconoscimento ("feature") tramite tre file di testo in /etc/appfilter/: feature.cfg (file effettivamente caricato dal demone all'avvio), feature_en.cfg (set di firme in inglese) e feature_cn.cfg (set di firme in cinese, copiato automaticamente su feature.cfg al primo avvio se quest'ultimo non esiste, come da script /etc/init.d/appfilter).

Ricerca delle regole di interesse nei tre file:

```
grep -i google /etc/appfilter/feature_en.cfg  
grep -iE "apple|appstore|itunes|icloud" /etc/appfilter/feature.cfg
```

Risultato: entrambi i set di firme contengono già regole pronte per gli store di interesse, individuate dai rispettivi ID numerici (appid):

- 7001 GooglePlay — match su dominio play.google.com (presente solo in feature_en.cfg)
- 7002 AppStore — match su dominio itunes.apple.com (presente in feature.cfg, quindi già nel set attivo di default)

La regola AppStore punta al dominio itunes.apple.com, mentre il traffico moderno dell'App Store passa in gran parte per apps.apple.com. È possibile che questa firma, risalente presumibilmente a una versione meno recente dell'App Store, non intercetti tutto il traffico reale.

12.2 Unione delle regole nel file attivo

Poiché il file attivo (feature.cfg, derivato dal set cinese) non conteneva la regola GooglePlay, è stata copiata dal set inglese. Un primo tentativo di append con grep "^7001" ... >> feature.cfg ha prodotto una riga corrotta, per l'assenza di un carattere di fine riga (a capo) alla fine del file di destinazione: la nuova riga si è concatenata alla fine dell'ultima regola esistente, danneggiandola. Il file è stato quindi rigenerato da zero e la regola aggiunta correttamente, garantendo un a capo prima dell'inserimento:

```
cp /etc/appfilter/feature_cn.cfg /etc/appfilter/feature.cfg  
echo "" >> /etc/appfilter/feature.cfg  
grep "GooglePlay" /etc/appfilter/feature_en.cfg >> /etc/appfilter/feature.cfg
```

Verifica dell'esito, con conferma delle tre regole distinte e integre (GooglePlay, AppStore, e la regola SSH preesistente, inizialmente danneggiata dal primo tentativo):

```
grep -E "GooglePlay|AppStore|SSH" /etc/appfilter/feature.cfg  
# 7002 AppStore:[tcp;;;itunes.apple.com;;] HIDE:0  
# 7035 SSH:[tcp;;;00:53|01:53|02:48]  
# 7001 GooglePlay:[tcp;;;play.google.com;;]
```

File feature.cfg aggiornato correttamente con le regole GooglePlay (7001) e AppStore (7002) attive.

13. Fase 9 — Ricostruzione dell'architettura di configurazione di OAF

L'attivazione del filtro globale (`uci set appfilter.global.enable='1'`) non è risultata sufficiente a rendere operative le regole: nei log compariva ripetutamente l'errore `oafd[...]: uci: Entry not found`, segno che il demone si aspettava di trovare dati in sezioni UCI (`rule`, `af_user`) risultate vuote nella configurazione di default.

Non essendo disponibile una documentazione ufficiale dettagliata sulla sintassi runtime, l'architettura reale è stata ricostruita per ispezione diretta dei binari e degli script di sistema, invece di procedere per tentativi:

```
uci show appfilter
strings /usr/bin/oafd | grep -iE "mac|appid|group|policy|action" | sort -u
find / -iname "*oaf*" 2>/dev/null
cat /usr/bin/oaf_rule
```

Quest'ultimo comando si è rivelato risolutivo: `/usr/bin/oaf_rule` non è un binario compilato ma uno script di shell, leggibile per intero, che rappresenta la vera interfaccia tra la configurazione UCI e il modulo kernel.

13.1 Architettura di funzionamento ricostruita

Dalla lettura dello script emerge il seguente flusso operativo:

- Le impostazioni globali (`work_mode`, `user_mode`, `app_filter_mode`, `lan_ifname`, `enable`, `record_enable`, ecc.) vengono lette da UCI e scritte come semplici file di testo dentro l'interfaccia virtuale del modulo kernel, esposta in `/proc/sys/oaf/` (uno pseudo-file per parametro, es. `/proc/sys/oaf/enable`, `/proc/sys/oaf/user_mode`)
- Le regole applicative vere e proprie, quali appid filtrare (sezione UCI `appfilter.rule`, campo `app_list`) e quali dispositivi coinvolgere per MAC address (sezione UCI `appfilter.af_user`, lista di sotto-sezioni anonime ciascuna con un campo `mac`), vengono invece serializzate in formato JSON e scritte nel device speciale `/dev/appfilter`, non in `/proc/sys/oaf`
- Esiste inoltre una sezione `whitelist` (anch'essa basata su MAC) per escludere dispositivi dal filtro quando il sistema opera in modalità automatica
- Il comando `/usr/bin/oaf_rule reload` è il punto di applicazione: legge la configurazione UCI corrente e la traduce nelle scritture sopra descritte (`/proc/sys/oaf/*` e `/dev/appfilter`). Senza questo comando, le modifiche fatte con `uci set/uci commit` restano salvate nel file di configurazione ma non vengono mai comunicate al modulo kernel attivo, spiegazione più probabile dell'errore `uci: Entry not found` osservato nei log, generato da un tentativo di reload automatico su una configurazione incompleta

13.2 Parametri disponibili in `/proc/sys/oaf/`

Elenco completo dei parametri esposti dal modulo kernel, verificato con `ls -la /proc/sys/oaf/`: `enable`, `work_mode`, `user_mode`, `app_filter_mode`, `lan_ifname`, `lan_ip`, `lan_mask`, `record_enable`, `disable_quic`, `tcp_rst`, `by_pass_accl`, `debug`, `feature_init`, `test_mode`, `version` (quest'ultimo l'unico in sola lettura).

13.3 Applicazione della configurazione

Sulla base dell'architettura ricostruita, è stata applicata la configurazione UCI necessaria a rendere operative le regole di riconoscimento individuate (appid 7001 GooglePlay e 7002 AppStore), in modalità automatica (tutti i dispositivi connessi, nessuna whitelist esclusa):

```
# 1. Impostare le app da riconoscere nella sezione rule
uci delete appfilter.rule.app_list
uci add_list appfilter.rule.app_list='7001'
uci add_list appfilter.rule.app_list='7002'

# 2. Modalità automatica: tutti i dispositivi tranne eventuale whitelist
uci set appfilter.global.user_mode='0'

# 3. Conferma abilitazione filtro e registrazione
```



```
uci set appfilter.global.enable='1'
uci set appfilter.global.record_enable='1'
uci commit appfilter

# 4. Applicazione effettiva al modulo kernel (passaggio precedentemente mancante)
/usr/bin/oaf_rule reload

# 5. Verifica che i parametri siano stati scritti correttamente
cat /proc/sys/oaf/enable
cat /proc/sys/oaf/user_mode
```

Questa configurazione, applicata al termine della sessione di ricostruzione dell'architettura, è risultata poi già presente e operativa nella sessione successiva, applicata autonomamente anche dall'interfaccia LuCI una volta installata, a conferma indiretta che le due strade (comandi UCI diretti e interfaccia grafica) agiscono sulle medesime sezioni di configurazione.

14. Fase 10 — Installazione dell'interfaccia grafica LuCI

Per facilitare la configurazione e la consultazione dei dati raccolti da OAF, è stata installata l'interfaccia web LuCI, inizialmente assente su questa immagine (non inclusa nella lista pacchetti originale dell'Image Builder).

```
apk add luci uhttpd
/etc/init.d/uhttpd enable
/etc/init.d/uhttpd start
cd /tmp
wget -O luci-app-oaf-6.1.4-r1.apk
https://github.com/destan19/OpenAppFilter/releases/download/v6.1.8/luci-app-oaf-6.1.4-r1.apk
apk add --allow-untrusted luci-app-oaf-6.1.4-r1.apk
/etc/init.d/uhttpd restart
```

Verifica dell'esito: interfaccia raggiungibile da un client connesso a TestAP all'indirizzo <http://192.168.1.1>, con menu dedicato ad AppFilter contenente tre viste principali: Device Info (elenco dispositivi connessi con relative app riconosciute), App Statistics (tempo di utilizzo aggregato per app, visualizzato a grafico) e Access Records (log dettagliato delle sessioni con timestamp di inizio/fine e stato del filtro).

Interfaccia LuCI per OAF installata e funzionante.

15. Fase 11 — Applicazione della configurazione e primo test con traffico reale

La configurazione UCI pianificata nella sessione precedente (sezione `appfilter.rule.app_list` con gli appid 7001/7002, `appfilter.global.user_mode='0'` per la modalità automatica) è risultata già presente nel sistema al momento della ripresa dei lavori, applicata autonomamente dall'interfaccia LuCI durante l'esplorazione iniziale della stessa, a conferma che l'interfaccia grafica scrive nelle medesime sezioni UCI individuate manualmente nella sessione precedente.

Primo test: un iPhone (dispositivo di test, distinto dall'iPhone 17 usato come sorgente di tethering/hotspot) è stato connesso alla rete TestAP e utilizzato per navigare nell'App Store. La tabella Device Info di LuCI ha mostrato immediatamente il riconoscimento dell'app (icona App Store nella colonna Common App), confermato in dettaglio nella vista Access Records:

- App Name: AppStore
- Start Visit Time: 27/07/2026, 15:25:36 — Last Visit Time: 27/07/2026, 15:32:53
- Filter Status: Filtered

La vista App Statistics ha inoltre mostrato un'aggregazione temporale coerente (6 minuti totali di utilizzo rilevato).

Riconoscimento del traffico App Store confermato su traffico reale, con evidenza puntuale (timestamp, durata, stato del filtro).

16. Fase 12 — Accesso ai dati di riconoscimento da riga di comando e individuazione di un conflitto di ID

16.1 Ricerca dei dati di riconoscimento accessibili da terminale

I dati mostrati da LuCI nelle viste Access Records e App Statistics non risultavano visibili tramite il log di sistema standard:

```
logread | grep -iE "GooglePlay|AppStore" | tail -20
```

Il comando non restituiva alcun risultato. Questo non costituisce un errore: i dati osservabili in LuCI evidentemente non passano dal log di sistema (syslog/logread), ma vengono gestiti dal demone oafd tramite una propria struttura dati interna, consultata da LuCI attraverso ubus, il bus di comunicazione interno di OpenWrt tipicamente usato per l'interrogazione di dati "vivi" tra demoni e interfaccia. È stata quindi condotta una ricerca mirata per individuare dove tali dati fossero effettivamente conservati e accessibili:

```
find /tmp /var -iname "*oaf*" 2>/dev/null
```

Risultato:

```
/tmp/luci-app-oaf-6.1.4-r1.apk
/tmp/.ujailnoafile
/tmp/log/oaf_luci.log
```

Il file /tmp/log/oaf_luci.log è risultato essere un log applicativo minimale, contenente solo le richieste effettuate dall'interfaccia LuCI stessa (non i dati di traffico):

```
cat /tmp/log/oaf_luci.log
# [2026-07-27 13:33:23] get class list
# [2026-07-27 13:37:04] get class list
```

È stata quindi verificata la disponibilità di un'interfaccia ubus dedicata:

```
ubus list | grep -i app
```

Risultato positivo: individuato l'oggetto appfilter.

Interrogazione dell'elenco completo dei metodi esposti:

```
ubus -v list appfilter
```

L'oggetto espone un'ampia API, di cui si riportano i metodi individuati come rilevanti per l'estrazione dei dati di riconoscimento (accanto ad altri dedicati alla configurazione, non utilizzati in questa fase):

- dev_list — elenco dei dispositivi connessi con le rispettive app riconosciute (corrisponde alla vista Device Info di LuCI)
- dev_visit_list — presumibilmente il dettaglio delle sessioni per dispositivo (vista Access Records)
- dev_visit_time — tempi aggregati di utilizzo (vista App Statistics)
- app_class_visit_time — tempo aggregato per categoria/app
- class_list — elenco delle categorie/app riconoscibili dal sistema

Il metodo dev_list è stato interrogato con successo:

```
ubus call appfilter dev_list
```

Il comando restituisce un oggetto JSON con, per ciascun dispositivo connesso, l'elenco delle app riconosciute (appid e nome), indirizzo MAC, IP, hostname e stato online, confermando la disponibilità di un canale diretto, scriptabile, per l'estrazione dei dati di riconoscimento senza dipendere dall'interfaccia grafica. I

metodi `dev_visit_list`, `dev_visit_time` e `app_class_visit_time` restano da esplorare nella prossima sessione di lavoro, per verificare se offrono dati più granulari (in particolare i timestamp puntuali già osservati in LuCI).

16.2 Individuazione di un conflitto di ID tra regole

Interrogando `dev_list` su traffico reale, è stata rilevata un'anomalia: tra le app riconosciute sul dispositivo "Giulia" (laptop) compariva una voce con id 7001 e name "迅雷" (Xunlei, un download manager cinese) anziché GooglePlay, nonostante l'ID 7001 fosse quello assegnato a GooglePlay nella sessione precedente.

Diagnosi: il file `feature.cfg` attivo, derivato dal set cinese (`feature_cn.cfg`) e integrato con la regola GooglePlay copiata dal set inglese, conteneva due regole distinte con lo stesso identificativo numerico (7001), un conflitto introdotto inconsapevolmente durante l'unione dei due file nella sessione precedente, poiché le numerazioni dei due set di firme non sono reciprocamente coordinate.

```
grep -n "^7001" /etc/appfilter/feature.cfg
# 141:7001 迅雷:[...]
# 236:7001 GooglePlay:[tcp;;;play.google.com;;]
```

Verifica dell'assenza di altri conflitti nel file, e individuazione dell'ID più alto in uso per sceglierne uno libero da assegnare a GooglePlay:

```
grep -oE "[0-9]+" /etc/appfilter/feature.cfg | sort -n | uniq -d
# 7001 (unico duplicato)
grep -oE "[0-9]+" /etc/appfilter/feature.cfg | sort -n | tail -5
# ... 8076 (id più alto in uso)
```

Correzione: riassegnazione della regola GooglePlay a un nuovo ID (9001), scelto ben al di sopra degli ID esistenti per escludere futuri conflitti, seguita dall'aggiornamento della configurazione UCI e dalla riapplicazione al modulo kernel:

```
sed -i '236s/^7001/9001/' /etc/appfilter/feature.cfg
uci del_list appfilter.rule.app_list='7001'
uci add_list appfilter.rule.app_list='9001'
uci commit appfilter
/usr/bin/oaf_rule reload
```

Verifica dell'esito: `uci show appfilter.rule` conferma la configurazione corretta (`appfilter.rule.app_list='7002' '9001'`), senza più riferimenti al vecchio ID in conflitto.

Nota metodologica: questo problema non era rilevabile dalla sola ispezione del file `feature.cfg` subito dopo la modifica (la riga GooglePlay era presente, sintatticamente corretta, e non generava errori di parsing), è emerso solo osservando un comportamento anomalo nei dati reali restituiti dal sistema (il nome errato associato a un ID), a conferma dell'importanza di validare empiricamente ogni integrazione tra fonti di configurazione eterogenee, non solo verificarne la sintassi.

Conflitto di ID risolto. Configurazione `appfilter.rule.app_list='7002' '9001'` applicata e verificata.

17. Fase 13 — Indagine sul mancato completamento dei download App Store

Durante un nuovo ciclo di test è stato osservato che l'App Store, sull'iPhone di test connesso a TestAP, permetteva la navigazione ma non il completamento del download di un'applicazione (il processo si avviava ma non terminava mai). Lo stesso dispositivo, connesso a una rete Wi-Fi diversa, completava regolarmente i download, isolando il problema alla rete TestAP e non al dispositivo.

17.1 Esclusione delle ipotesi di rete di base

Sono state verificate ed escluse, in ordine, le seguenti ipotesi:

- Frammentazione/MTU: l'analisi di una cattura tcpdump (sia lato LAN che lato WAN) durante un tentativo di download non ha mostrato segnali di pacchetti scartati, ICMP di frammentazione, o

connessioni interrotte a metà, i flussi TCP osservati si completavano regolarmente, anche con pacchetti superiori a 1300 byte. Il parametro `mtu_fix` del firewall è risultato già correttamente abilitato.

- Versione iOS incompatibile: esclusa, poiché lo stesso dispositivo scarica regolarmente su altre reti.
- Connettività IPv6 parziale sul tethering: individuato un problema reale (l'interfaccia `eth1` esponeva solo un indirizzo IPv6 link-local, senza rotta verso Internet, mentre il DNS continuava a fornire ai client anche risposte IPv6 per i domini Apple, inducendo tentativi di connessione falliti secondo il comportamento standard Happy Eyeballs di iOS). Corretto disabilitando le risposte AAAA sul resolver locale (`dhcp.@dnsmasq[0].filter_aaaa='1'`). La correzione risulta tecnicamente valida e verificata (`nslookup` restituisce ora solo indirizzi IPv4), ma non ha, da sola, risolto il problema del download.

17.2 Evidenza del filtro attivo su tutte le sessioni App Store

Consultando la vista Access Records di LuCI per il dispositivo di test, è stato osservato un pattern sistematico: ogni sessione App Store risulta marcata con Filter Status "Filtered", in tutte le occorrenze registrate nella giornata, mentre altro traffico di sottofondo (riconosciuto come app estranee al contesto, es. 陌陌/Momo e 淘宝/Taobao, verosimilmente per sovrapposizione di domini/CDN) risulta "Unfiltered":

Device Details (iPhone)

App Statistics		Access Records		
App Name	Start Visit Time	Last Visit Time	Duration	Filter Status
 AppStore	28/07/2026, 15:54:49	28/07/2026, 16:03:08	-	Filtered
 陌陌	28/07/2026, 16:01:03	28/07/2026, 16:01:03	1m	Unfiltered
 AppStore	28/07/2026, 15:40:15	28/07/2026, 15:48:35	-	Filtered
 AppStore	28/07/2026, 14:59:44	28/07/2026, 15:32:59	-	Filtered
 淘宝	28/07/2026, 15:13:13	28/07/2026, 15:13:13	1m	Unfiltered

Tabella riassuntiva delle sessioni osservate:

App Name	Start Visit Time	Last Visit Time	Duration	Filter Status
AppStore	28/07/2026, 15:54:49	28/07/2026, 16:03:08	-	Filtered
陌陌 (Momo)	28/07/2026, 16:01:03	28/07/2026, 16:01:03	1m	Unfiltered
AppStore	28/07/2026, 15:40:15	28/07/2026, 15:48:35	-	Filtered
AppStore	28/07/2026, 14:59:44	28/07/2026, 15:32:59	-	Filtered
淘宝 (Taobao)	28/07/2026, 15:13:13	28/07/2026, 15:13:13	1m	Unfiltered

Osservazione: la sistematicità del Filter Status "Filtered" su tutte e tre le sessioni App Store, registrate in momenti diversi della giornata, riapre il sospetto, inizialmente scartato sulla base di un test con `appfilter.global.enable='0'` che non era stato verificato in modo rigoroso subito prima del tentativo di download che sia effettivamente OAF, tramite il meccanismo `tcp_rst` (interruzione forzata della connessione via TCP reset), a impedire il completamento del download, indipendentemente dal problema IPv6 corretto in parallelo.

17.3 Conferma della causa e bug individuato nello script `oaf_rule`

Il test pianificato è stato eseguito: verifica esplicita del valore effettivo in `/proc/sys/oaf/enable` prima di procedere.

```
uci show appfilter.global | grep -E "enable|tcp_rst"
# appfilter.global.enable='0' (impostato in una sessione precedente)
cat /proc/sys/oaf/enable
# 1 (valore reale nel modulo kernel, discordante da UCI)
```

È stata individuata la causa della discordanza: la funzione `reload_base_config()` dello script `/usr/bin/oaf_rule` scrive su `/proc/sys/oaf/` solo i parametri `work_mode`, `user_mode`, `app_filter_mode` e `lan_ifname`, non `enable`, né `tcp_rst`. Di conseguenza, modificare questi due parametri da UCI e lanciare `oaf_rule reload` non ha alcun effetto sul modulo kernel attivo: si tratta di una lacuna dello script ufficiale, non di un errore di configurazione. Ne consegue che tutti i tentativi precedenti di disattivare il filtro tramite UCI + reload, non erano mai stati realmente applicati.

Correzione tramite scrittura diretta sul pseudo-file esposto dal modulo kernel, bypassando lo script:

```
echo 0 > /proc/sys/oaf/enable
cat /proc/sys/oaf/enable
# 0 (confermato)
```

Test di conferma, ripetuto con due dispositivi distinti (l'iPhone di test e uno smartphone di un'altra persona, per escludere cause specifiche del singolo dispositivo): con `/proc/sys/oaf/enable` a 0, il download di un'applicazione dall'App Store si completa correttamente. Riportando `enable` a 1 e disattivando invece selettivamente solo `tcp_rst`, il download continua a completarsi correttamente:

```
echo 1 > /proc/sys/oaf/enable
echo 0 > /proc/sys/oaf/tcp_rst
```

Questo isola con certezza la causa: il parametro `tcp_rst` (interruzione forzata della connessione tramite TCP reset all'atto del riconoscimento di una regola) era responsabile del mancato completamento dei download, indipendentemente dal problema di connettività IPv6 individuato in precedenza (che pure costituiva un problema reale, solo non la causa principale del sintomo osservato).

Nota metodologica per la tesi: questo episodio evidenzia un limite pratico non banale nell'uso di OAF, l'interfaccia UCI espone parametri che, per un bug dello script di gestione fornito dal progetto, non vengono effettivamente applicati al modulo kernel tramite il consueto meccanismo reload. La sola ispezione della configurazione dichiarata (`uci show`) non è quindi sufficiente a determinare lo stato di funzionamento reale del sistema: è necessario verificare lo stato effettivo esposto direttamente dal modulo kernel (`/proc/sys/oaf/*`).

Causa del blocco download confermata: parametro `tcp_rst`. Configurazione operativa individuata (`enable=1`, `tcp_rst=0`): riconoscimento attivo, blocco disattivato

17.4 Revisione della diagnosi: instabilità di `oafd` e causa reale isolata

La configurazione individuata (`enable=1`, `tcp_rst=0`, scritti direttamente su `/proc/sys/oaf/`) si è rivelata, in una sessione di lavoro successiva, non affidabile: i valori risultavano soggetti a modifiche periodiche non richieste esplicitamente, tornando spontaneamente ai valori precedenti nell'arco di pochi secondi.

```
for i in 1 2 3 4 5; do cat /proc/sys/oaf/enable; sleep 3; done
# 1
# 1
# 0
# 0
# 0
```

È stato individuato il responsabile: il demone `oafd` stesso, verosimilmente dotato di un ciclo di sincronizzazione interno non documentato, riscrive periodicamente i parametri del modulo kernel indipendentemente da modifiche dirette su `/proc/sys/oaf/` o dalla configurazione UCI dichiarata (verificata correttamente impostata tramite `uci get`, a conferma che la discrepanza non è imputabile a un errore di configurazione).

Arrestando il demone, i valori sono risultati stabili nel tempo:

```
/etc/init.d/appfilter stop
echo 1 > /proc/sys/oaf/enable
echo 0 > /proc/sys/oaf/tcp_rst
for i in 1 2 3 4 5 6; do cat /proc/sys/oaf/enable; sleep 3; done
# 1 (stabile per tutta la durata del test)
```

Tuttavia, con il demone fermo, l'interfaccia ubus (necessaria per il riconoscimento/logging) risulta non disponibile:

```
ubus call appfilter dev_list
# Command failed: Not found
```

Un tentativo di download con il demone fermo (quindi con enable=1 e tcp_rst=0 stabili, ma privo di riconoscimento attivo) è comunque fallito, indicando che il blocco persisteva a livello di stato interno del modulo kernel, non riconducibile né al demone né al valore istantaneo di tcp_rst. Il problema è stato risolto inviando esplicitamente un comando di reset delle regole al device di comunicazione del modulo:

```
echo '{"op":3,"data":{}}' > /dev/appfilter
```

Dopo questo comando, il download si è completato con successo. Riavviando il demone (/etc/init.d/appfilter start) e ripristinando le regole tramite oaf_rule reload, il download è tornato a fallire, nonostante enable e tcp_rst fossero confermati rispettivamente a 1 e 0 in modo stabile. Isolando la variabile tramite lo svuotamento esplicito della lista delle app filtrate (mantenendo il demone attivo):

```
uci set appfilter.rule.app_list=''
uci commit appfilter
/usr/bin/oaf_rule reload
```

il download è tornato a funzionare correttamente, con il demone regolarmente attivo.

Il blocco del download non dipende dal parametro tcp_rst, che era stato erroneamente identificato come causa a seguito di una sessione di test compromessa dalla instabilità di oafd sopra descritta. La causa reale è la sola presenza dell'app nella lista appfilter.rule.app_list: OAF, nella configurazione di default utilizzata (app_filter_mode a specified apps), sembra bloccare intrinsecamente le app elencate in tale lista per i dispositivi in ambito, indipendentemente dal valore di tcp_rst, parametro che ne controlla solo la modalità di interruzione della connessione (TCP reset esplicito vs. altro meccanismo), non se il blocco avvenga.

L'architettura di OAF, così come osservata, non sembra offrire nativamente una modalità di puro monitoraggio (riconoscimento senza blocco) per le app esplicitamente elencate nelle regole. Questo è un vincolo pratico rilevante da riportare: per raccogliere dati di traffico completi sarà necessario un protocollo di test che alterni fasi con la regola attiva (per registrare l'evento di riconoscimento) a fasi con la regola rimossa (per lasciar completare il traffico osservato), oppure verificare se un parametro non ancora individuato consenta di disaccoppiare le due funzioni.

Causa reale del blocco isolata: presenza dell'app nella lista appfilter.rule.app_list, non tcp_rst. Comportamento di oafd (riscrittura periodica dei parametri) documentato. Protocollo di test per la raccolta dati ancora da definire.

18. Fase 14 — Comportamento dello stato online/offline dei dispositivi

Durante l'osservazione della vista Device Info in LuCI è stata notata un'apparente incongruenza: un dispositivo di test risultava ancora in stato "Online" pur essendo, di fatto, in standby (schermo spento, nessuna app aperta in primo piano) da diversi minuti.

Verifica del lease DHCP attivo per il dispositivo in questione:

```
cat /tmp/dhcp.leases
```

Risultato: il MAC del dispositivo risultava ancora presente con un lease valido, confermando che lo stato "Online" riportato da OAF riflette la persistenza della connessione di rete (associazione Wi-Fi/lease DHCP attivo), non un'attività di traffico effettivamente in corso in quel momento.

Conferma incrociata tramite `ubus`, verificando che l'elenco delle app riconosciute per il dispositivo non fosse cresciuto rispetto all'osservazione precedente, nonostante il dispositivo fosse fermo:

```
ubus call appfilter dev_list
```

La `applist` del dispositivo è risultata invariata, a conferma che il riconoscimento delle app riflette correttamente solo l'attività di traffico reale, mentre lo stato online/offline è un indicatore di connessione di rete, non di attività applicativa: i due dati non vanno confusi nell'interpretazione dei risultati raccolti.

Osservazione aggiuntiva: il passaggio dallo stato Online a Offline non è risultato immediato all'interruzione dell'attività, ma ritardato di alcuni minuti. Una ricerca nel codice del demone conferma l'esistenza di un meccanismo di timeout dedicato, distinto dal lease time DHCP (verificato pari a 12h, quindi non correlato):

```
uci show dhcp | grep -i leasetime
# dhcp.lan.leasetime='12h'

strings /usr/bin/oafd | grep -iE "timeout|expire|online"
# dev:%s expired, offline time = %ds, count=%d, visit_count=%d
# ...
```

Il valore esatto del timeout applicato da OAF non è stato individuato con precisione (verosimilmente un valore non esposto come stringa leggibile nel binario). Non è stato approfondito ulteriormente in questa fase, non essendo bloccante per il proseguimento del lavoro; resta un'osservazione da tenere presente nella raccolta dati: il momento in cui un dispositivo viene marcato offline non coincide esattamente con il momento in cui ha smesso di generare traffico, per cui il riferimento temporale affidabile per isolare le sessioni resta il timestamp puntuale riportato in Access Records, non lo stato online/offline generico.

19. Fase 15 — Analisi del log dettagliato e conferma del meccanismo di blocco

Per comprendere con precisione il meccanismo che causa il fallimento del download, è stato attivato il livello di debug massimo del modulo kernel e osservato il log in tempo reale durante un tentativo di download reale, con la sola regola AppStore (7002) attiva:

```
echo 2 > /proc/sys/oaf/debug
logread -f
```

Estratto significativo del log raccolto durante il tentativo (timestamp reali, dispositivo iPhone di test, IP 192.168.1.131):

```
match tcp 192.168.1.131(51161)--> 23.7.245.218(443) len = 1400, 7002
match ct client hello...
match https host failed, data_len = 74, sport:35904, dport:443
match feature, appid=7002, feature = tcp;;;itunes.apple.com;;
match tcp 192.168.1.131(51162)--> 23.7.245.220(443) len = 1400, 7002
...
match feature, appid=7002, feature = tcp;;;itunes.apple.com;;
match tcp 192.168.1.131(51168)--> 23.7.245.220(443) len = 1400, 7002
match tcp 192.168.1.131(51169)--> 23.7.245.218(443) len = 1400, 7002
```

Osservazione chiave: nel corso di un singolo tentativo di download sono state osservate almeno quattro connessioni TCP distinte, verso indirizzi IP diversi (tutti appartenenti alla rete Akamai, la CDN utilizzata da Apple), tutte riconosciute indipendentemente come appid 7002 perché ciascuna presenta `itunes.apple.com` nel campo SNI (Server Name Indication) del proprio TLS ClientHello, individuato dalla funzione `dpi_https_proto` del modulo (visibile nel log come "match ct client hello").

Questo smentisce l'ipotesi formulata inizialmente secondo cui `itunes.apple.com` fosse solo un dominio di "passaggio" iniziale (autenticazione/redirect) prima del vero trasferimento dati su altri domini. I dati mostrano invece che una parte sostanziale del traffico reale del download, in questa versione di iOS/App Store, utilizza ripetutamente connessioni con SNI `itunes.apple.com`. Di conseguenza, quando la regola 7002 provoca l'interruzione di queste connessioni (verosimilmente tramite DROP a livello netfilter, dato che il

comportamento persiste anche con tcp_rst=0), viene colpito traffico funzionalmente necessario al completamento del download, non solo un passaggio accessorio.

19.1 Un ulteriore dato utile individuato nel log: contatori di traffico per sessione

Nel medesimo log è stata individuata una riga di report periodico generata dal modulo kernel, contenente contatori di traffico aggregati per dispositivo e sessione:

```
report:{"mac":"3a:0f:fd:eb:6f:43","ip":"192.168.1.131","app_num":0,
"up_flow":261,"down_flow":391,"active":1,
"visit_info":[{"appid":7002,"latest_action":0}]} count=2
```

I campi up_flow e down_flow rappresentano presumibilmente il volume di byte in upload/download associato alla sessione.

20. Fase 16 — Protocollo di raccolta dati ad alternanza

Poiché OAF, con la configurazione osservata, non consente di disaccoppiare riconoscimento e blocco per le app esplicitamente elencate in appfilter.rule.app_list, è stato definito un protocollo operativo a due fasi alternate, da utilizzare per ogni singola osservazione della campagna di raccolta dati:

20.1 Fase A — Registrazione dell'evento (regola attiva)

La regola relativa allo store di interesse viene attivata, e il dispositivo di test viene indotto a iniziare l'azione da osservare (apertura dello store, ricerca di un'applicazione). L'obiettivo di questa fase non è completare un download, ma catturare l'evento di riconoscimento con il relativo timestamp preciso, tramite interrogazione dell'interfaccia ubus:

```
uci set appfilter.rule.app_list='7002'
uci add_list appfilter.rule.app_list='9001'
uci commit appfilter
/usr/bin/oaf_rule reload
# [avviare l'azione sul dispositivo di test]
ubus call appfilter dev_list
```

20.2 Fase B — Completamento del traffico (regola rimossa)

Subito dopo aver registrato l'evento, la regola viene rimossa per consentire il completamento reale del traffico (es. il download effettivo di un'applicazione), necessario per raccogliere un campione di traffico completo da analizzare:

```
uci set appfilter.rule.app_list=''
uci commit appfilter
/usr/bin/oaf_rule reload
# [il traffico completa liberamente sul dispositivo di test]
```

20.3 Considerazioni pratiche per l'esecuzione del protocollo

- Le due fasi vanno eseguite in sequenza ravvicinata per ogni singola osservazione, non una sola volta per l'intera sessione di test.
- Prima di ogni ripetizione, applicare il reset dello stato del dispositivo già pianificato in precedenza (disinstallazione dell'app, pulizia cache dello store), per garantire che ogni osservazione sia indipendente dalle precedenti.
- Il MAC address del dispositivo iOS di test può variare tra una connessione e l'altra per effetto della randomizzazione (Private Wi-Fi Address); per identificare in modo affidabile lo stesso dispositivo nel tempo, valutare la disattivazione di tale funzione per la rete TestAP specifica.
- Ripetere ogni scenario (ogni singola applicazione scelta per il test) un numero sufficiente di volte (indicativamente 5-6 ripetizioni) prima di trarre conclusioni sulla distinguibilità dei pattern di traffico.

- Affiancare, quando possibile, una cattura tcpdump sull'interfaccia br-lan durante la Fase B, per disporre sia dei contatori aggregati offerti da OAF sia del dettaglio pacchetto per pacchetto, utile per un'analisi più fine (es. sequenza delle dimensioni dei primi pacchetti, come nell'approccio dei lavori di riferimento consultati in fase di proposta).

Causa del blocco chiarita in dettaglio (SNI itunes.apple.com presente su più connessioni del reale traffico di download, non solo su un passaggio iniziale). Protocollo di raccolta dati ad alternanza definito e pronto per la prima campagna di test dello Stadio 2.

21. Fase 17 — Prima campagna di raccolta dati (classificazione_app)

In questa sessione è stata condotta la prima vera campagna di raccolta dati secondo il protocollo ad alternanza, applicandolo a un confronto tra applicazioni di dimensione crescente. La sessione ha attraversato numerose difficoltà tecniche impreviste, in particolare legate alla stabilità del sistema sotto carico sostenuto, che hanno richiesto diverse soluzioni correttive documentate di seguito.

21.1 Script di automazione del ciclo di test

È stato creato uno script (/root/test_cycle.sh) che automatizza l'intero protocollo ad alternanza, con checkpoint interattivi per sincronizzare le azioni sul dispositivo di test con le fasi di attivazione/rimozione della regola OAF:

```
#!/bin/sh
APP=$1
TS=$(date +%Y%m%d_%H%M%S)
PCAP="/mnt/usb/cattura_${APP}_${TS}.pcap"
LOG="/mnt/usb/log_${APP}_${TS}.txt"

# FASE A: attivo il blocco
uci set appfilter.rule.app_list='7002'
uci add_list appfilter.rule.app_list='9001'
uci commit appfilter
/usr/bin/oaf_rule reload
# [attesa interattiva: ricerca app sul dispositivo]
ubus call appfilter dev_list > "$LOG"

# FASE B: rimuovo il blocco, avvio la cattura
uci delete appfilter.rule.app_list
uci commit appfilter
/usr/bin/oaf_rule reload
tcpdump -i br-lan -w "$PCAP" &
# [attesa interattiva: download sul dispositivo]
kill $TCPDUMP_PID
```

Uso: ./test_cycle.sh <nome_test>, dove ogni invocazione genera una coppia di file (cattura .pcap e log di riconoscimento .txt) con timestamp univoco, permettendo ripetizioni multiple senza sovrascritture.

21.2 Esaurimento dello spazio disco sulla root partition

La root partition del sistema si è rivelata di dimensione molto contenuta (98.3 MB totali). Salvando le catture di traffico direttamente in /root, lo spazio si è esaurito dopo circa 60 MB di catture accumulate (tre ripetizioni di Calcolatrice più due di Disney+), causando errori di scrittura a catena su qualunque file, incluso il file di configurazione di OAF e una cattura andata perduta (0 byte).

```
df -h
# /dev/root    98.3M   96.3M    0   100%   /
```

Soluzione adottata: collegamento di una chiavetta USB esterna (28.9 GB, filesystem NTFS) come destinazione per le catture, montata manualmente in /mnt/usb. È stato necessario installare il modulo del driver NTFS non presente di default sull'immagine:

```
apk add kmod-fs-ntfs3
mount -t ntfs3 /dev/sda1 /mnt/usb
```

Lo script è stato aggiornato per scrivere le catture sulla nuova destinazione, risolvendo temporaneamente il problema di spazio.

21.3 Instabilità del sistema sotto carico sostenuto: crash ricorrenti

Durante il tentativo di catturare il download di un'applicazione di grandi dimensioni (Genshin Impact, 3.5 GB dichiarati), il sistema ha smesso di rispondere e si è riavviato autonomamente dopo aver catturato circa 930 MB di traffico. Il fenomeno si è ripetuto più volte nel corso della sessione, con pattern diversi:

Tentativo	Destinazione scrittura	Esito
Genshin Impact (3.5 GB)	Chiavetta USB (NTFS)	Crash dopo ~930 MB catturati
Trasferimento SSH di un file da 124 MB già completo	N/A (sola lettura)	AP caduto durante il trasferimento
Hay Day (~700 MB), tentativo 1	Chiavetta USB (NTFS)	Crash dopo ~15 minuti, 162 MB catturati
Hay Day, tentativo 2 (con timeout di sicurezza 180s)	RAM (/tmp)	Crash dopo meno di 60 secondi
Canva (142.5 MB), con timeout attivo	RAM (/tmp)	Comando timeout+tcpdump non ha avviato la cattura (nessun file prodotto)
Canva, timeout rimosso	RAM (/tmp)	Riuscito, nessun crash (49.948 pacchetti, 0 persi)

Diagnosi condotta dopo i crash, tramite log di sistema al riavvio (il contenuto del buffer dmesg relativo al momento esatto del crash risulta perso a ogni riavvio, non essendo stato salvato su supporto persistente in tempo reale):

```
dmesg | grep -iE "over-current|USB disconnect"    # nessun risultato
dmesg | grep -iE "oom|out of memory"              # nessun risultato
free -h      # RAM totale 8 GB, quasi interamente libera
dmesg | grep -iE "watchdog"
# init: - watchdog - (righe presenti solo in fase di avvio, non conclusive)
```

Le cause più probabili (sovraccarico USB per alimentazione insufficiente, esaurimento memoria) sono state escluse dai controlli sopra. Un indizio quantitativo rilevante è stato calcolato sul tentativo Hay Day con 162 MB catturati in circa 15 minuti corrispondono a una velocità di scrittura media di soli ~180 KB/s, anomalamente bassa, **suggerendo il driver ntfs3 come possibile collo di bottiglia in scrittura sotto carico sostenuto**. Il passaggio a scrittura su RAM (/tmp, tmpfs) non ha tuttavia eliminato il problema in modo definitivo (crash anche su Hay Day tentativo 2, sebbene più rapido), indicando che la causa non è riconducibile alla sola velocità del supporto di scrittura, ma verosimilmente al **carico complessivo del sistema (NAT ad alto traffico, ispezione pacchetto-per-pacchetto da parte del modulo OAF, gestione simultanea di hostapd e tcpdump) su un hardware con risorse limitate** quando il volume di traffico intenso si protrae nel tempo.

Nota metodologica per la tesi: il limite pratico osservato empiricamente si colloca indicativamente tra 150 e 250 MB di cattura continua per le prove riuscite senza incidenti, con un margine di incertezza legato alla natura non completamente deterministica del fenomeno (stesso ordine di grandezza di traffico, esiti diversi tra un tentativo e l'altro). Questo è un dato rilevante da riportare come limite hardware del testbed, utile a giustificare lo scope delle applicazioni effettivamente testate per intero rispetto a quelle catturate solo parzialmente.

Causa dei crash non isolata con certezza assoluta, ma verosimilmente riconducibile al carico complessivo del sistema sotto traffico sostenuto e prolungato, non a un singolo fattore isolato (esclusi alimentazione USB e memoria). Soluzione pratica adottata: scrittura su RAM (/tmp) con trasferimento immediato dei dati su un supporto persistente al termine di ogni ciclo riuscito.

21.4 Metodo di trasferimento dati adottato

Per la messa in sicurezza dei dati raccolti sono stati impiegati, a seconda delle circostanze, tre metodi distinti: lettura via SSH con redirectione locale (per file di dimensione contenuta, già utilizzato nella tesi precedente), server web temporaneo tramite il servizio uhttpd già installato per LuCI (collegando simbolicamente la cartella di destinazione alla webroot), e rimozione fisica della chiavetta USB per il collegamento diretto a un PC (il metodo rivelatosi più affidabile per i file di dimensione maggiore, non soggetto ai limiti di banda/stabilità della rete Wi-Fi del testbed).

```
# Metodo 1: lettura via SSH
ssh root@192.168.1.1 "cat /tmp/cattura_X.pcap" > cattura_X.pcap

# Metodo 2: server web temporaneo
ln -s /mnt/usb /www/usb # poi: http://192.168.1.1/usb/

# Metodo 3: rimozione fisica della chiavetta
umount /mnt/usb # quindi disconnessione fisica e lettura diretta da PC
```

Nota aggiuntiva: in un'occasione, il filesystem NTFS della chiavetta è risultato non montabile in scrittura (mount: Invalid argument) in seguito a un riavvio non pulito del sistema durante una scrittura attiva. Il problema è stato diagnosticato come corruzione del journal NTFS (\$MFTMirr non corrispondente a \$MFT) e risolto con lo strumento di riparazione dedicato, mantenendo nel frattempo l'accesso in sola lettura per non perdere l'accesso ai dati già presenti:

```
mount -t ntfs3 -o ro /dev/sda1 /mnt/usb # accesso di emergenza in sola lettura
apk add ntfs-3g-utils
ntfsfix /dev/sda1
```

22. Fase 18 — Analisi quantitativa del dataset raccolto

Nei giorni successivi alla raccolta dati documentata, è stata condotta l'analisi quantitativa dei 54 file di cattura validi (9 applicazioni, 6 ripetizioni ciascuna), con l'obiettivo di verificare se il volume e le caratteristiche del traffico permettano di distinguere quale applicazione specifica è stata scaricata. L'analisi è stata condotta in un notebook Jupyter separato (eseguito localmente, non sul Raspberry Pi), popolato caricando i file .pcap trasferiti nelle sessioni precedenti.

22.1 Estrazione veloce delle statistiche

Un primo tentativo di estrazione con la libreria scapy (rdpcap, con decodifica completa di ogni pacchetto) è risultato troppo lento per le catture di grandi dimensioni, in particolare il campione parziale di Genshin Impact (878 MB, oltre 680.000 pacchetti), richiedendo diversi minuti solo per il caricamento in memoria.

È stata quindi implementata una lettura manuale del formato .pcap tramite la libreria struct, sfruttando la struttura fissa del formato (intestazione globale di 24 byte, seguita da record preceduti da un header di 16 byte con timestamp e lunghezza del pacchetto). Per le statistiche aggregate (numero di pacchetti, byte totali, durata) non è necessario decodificare il contenuto dei pacchetti: è sufficiente leggere questi header, saltando i dati veri e propri con un'operazione di seek. Questo approccio produce risultati numericamente identici a scapy, riducendo il tempo di esecuzione da minuti a pochi secondi anche sulle catture più pesanti.

La stessa tecnica è stata successivamente estesa per estrarre anche la direzione del traffico (byte upload/download), leggendo direttamente l'header IP (byte 14-34 del frame Ethernet) senza la decodifica completa di scapy.

22.2 Prima classificazione: feature di base

È stato costruito un classificatore Random Forest, validato con la tecnica leave-one-out (scelta necessaria data la ridotta numerosità campionaria: 6 ripetizioni per applicazione non permettono uno split train/test tradizionale senza rendere la stima dell'accuratezza altamente instabile). Le applicazioni Genshin Impact e Hay Day, disponendo di un solo campione ciascuna, sono state escluse dalla validazione del classificatore (incluse solo nell'analisi descrittiva), poiché la validazione leave-one-out non è significativa per classi con un solo esempio.

Con le quattro feature di base (MB_totali, durata_sec, rapporto_down_up, perc_pacchetti_pesanti, quest'ultima calcolata con una soglia di 1200 byte, scelta per coerenza con quella osservata nel codice sorgente del modulo kernel di OpenAppFilter) il classificatore ha raggiunto un'accuratezza del 79.6% su 9 classi (54 campioni), contro un livello atteso dell'11.1% per una scelta casuale.

L'analisi della matrice di confusione ha mostrato che gli errori si concentravano quasi esclusivamente tra applicazioni di dimensione simile (Canva, UPS, Unieuro, IKEA, tutte nella fascia 63-93 MB), mentre le applicazioni ben distanziate in volume (Calcolatrice, Corriere della Sera, Disney+, Shazam, Trivago) venivano classificate correttamente nella quasi totalità dei casi.

22.3 Tentativo con l'intero set di feature disponibili: risultato negativo

Nel tentativo di migliorare l'accuratezza, è stato costruito un set esteso di 13 feature (aggiungendo deviazione standard e percentili della dimensione dei pacchetti, statistiche sull'inter-arrival time (IAT) e il rapporto pacchetti/MB). L'accuratezza è però peggiorata a 70.4%, un risultato controintuitivo ma spiegabile: con soli 53-54 campioni di training per fold e 13 variabili, il modello dispone di un numero di feature elevato rispetto al numero di esempi, con un rischio concreto di overfitting su pattern che non generalizzano al campione escluso.

Risultato negativo ma metodologicamente rilevante: l'aggiunta indiscriminata di feature su un dataset di dimensioni ridotte peggiora le prestazioni. Necessaria una selezione mirata.

22.4 Selezione mirata delle feature: risultato migliore (83.3%)

Sulla base dell'analisi di importanza delle feature (attributo feature_importances_ del modello con le sole feature di base, in cui MB_totali risultava dominante seguita da iat_medio_ms), sono state selezionate solo due feature aggiuntive: il tempo medio di inter-arrivo tra pacchetti consecutivi (iat_medio_ms) e la deviazione standard della dimensione dei pacchetti (dim_std).

Con il set finale di 5 feature (MB_totali, rapporto_down_up, perc_pacchetti_pesanti, iat_medio_ms, dim_std) l'accuratezza è salita a 83.3%, il miglior risultato ottenuto in questo lavoro. Il miglioramento si è concentrato specificamente sulle applicazioni IKEA e Trivago, portate al 100% di recall (dall'83% precedente).

La rimozione della feature durata_sec dal modello (sostituita implicitamente dalle nuove feature) non ha modificato l'accuratezza, a conferma che il modello le attribuiva già un peso marginale, coerentemente con l'elevata variabilità di questa feature.

22.5 Verifica e confronto: iperparametri e XGBoost

Una ricerca sugli iperparametri del Random Forest (n_estimators, max_depth), validata con leave-one-out, ha confermato che la configurazione di default (n_estimators=200, profondità non limitata) era già ottimale: nessuna combinazione testata ha superato l'83.3% di accuratezza. Una prima ricerca più estesa (144 combinazioni) è stata scartata per il tempo di calcolo eccessivo (oltre 500 secondi) rispetto al beneficio atteso su un dataset di queste dimensioni.

Un secondo algoritmo, XGBoost, addestrato sullo stesso set di 5 feature con identica validazione leave-one-out, ha raggiunto la medesima accuratezza (83.3%), con una distribuzione degli errori leggermente diversa (Random Forest più accurato su IKEA e Trivago, XGBoost più accurato su UPS). La convergenza dei due algoritmi sullo stesso valore, unita alla persistenza dell'errore su Canva in entrambi i modelli, indica che il limite residuo non è attribuibile alla scelta dell'algoritmo di classificazione.

22.6 Analisi approfondita del caso residuo: Canva / UPS

Le applicazioni Canva e UPS restano confuse tra loro (50% di recall reciproco) anche dopo l'introduzione delle feature mirate. È stato condotto un confronto diretto dei valori delle feature (media e deviazione standard per gruppo), che ha rivelato un volume di traffico quasi indistinguibile tra le due applicazioni (Canva: 73.3 ± 11.1 MB; UPS: 70.2 ± 6.4 MB).

È stata inoltre osservata un'elevata variabilità interna della feature durata_sec per Canva (media 89.9s, deviazione standard 69.3s, quasi pari alla media): un singolo campione (canva_1) dura 189.6 secondi, mentre altri due (canva_5, canva_6) durano solo 20-21 secondi, un fattore di variazione di quasi 10 volte per la stessa identica applicazione. **Questo comportamento è stato attribuito a condizioni di rete non controllate al momento della cattura (congestione, velocità di connessione variabile nel momento del test), piuttosto che a un comportamento intrinsecamente diverso dell'applicazione.**

È stata scartata l'ipotesi di sostituire durata_sec con velocita_media_KB_s, poiché quest'ultima è matematicamente derivata da volume e durata (velocità = byte/tempo) ed erediterebbe lo stesso rumore legato alle condizioni di rete del momento, non al comportamento dell'applicazione.

Conclusione: la confusione residua tra Canva e UPS rappresenta un limite intrinseco del dataset raccolto, le due applicazioni generano volumi di traffico e pattern di pacchetti sufficientemente simili da non essere pienamente distinguibili con le feature e il numero di campioni disponibili in questo lavoro.

22.7 Esperimenti di classificazione precoce (primi N pacchetti)

Ispirandosi alla letteratura sulla classificazione precoce del traffico (**FastFlow**, e per l'approccio a sequenza di pacchetti, **AppScanner**), sono stati condotti due esperimenti aggiuntivi per verificare se sia possibile classificare un'applicazione osservando solo i primi N pacchetti, anziché l'intera sessione, requisito rilevante per un eventuale utilizzo in tempo reale, a differenza dell'analisi a posteriori condotta finora.

Approccio	Accuratezza (leave-one-out)	Esito
Feature aggregate, cattura intera	83.3%	Riferimento
Feature aggregate, primi N pacchetti (N=20...1000)	13-33%	Instabile, molto inferiore
Sequenza grezza (dimensione+direzione), primi N pacchetti (N=10...50)	~11% (livello casuale)	Nessun segnale utile

Il secondo risultato (sequenza grezza) è stato spiegato con il disallineamento temporale tra catture diverse della stessa applicazione: la posizione esatta in cui compare un dato pacchetto varia da una ripetizione all'altra per motivi legati alla rete (ritrasmissioni, timing di risposta del server), non all'applicazione stessa. Le feature aggregate, indipendenti dall'ordine esatto dei pacchetti, risultano quindi molto più robuste a questa variabilità, a differenza dell'approccio di FastFlow, che utilizza feature per finestra temporale invece che per pacchetto puntuale.

22.8 Sintesi dei risultati sullo Stadio 2 (distinzione tra applicazioni)

Esperimento	Accuratezza (leave-one-out)
Feature di base (4)	79.6%
+ ricerca iperparametri	79.6% (nessun miglioramento)
+ XGBoost al posto di Random Forest	81.5-83.3%
Set completo di 13 feature	70.4% (peggiora, overfitting)
Feature mirate (5): + IAT medio, dim_std	83.3% (risultato finale)

Il risultato finale (83.3%) va letto considerando la sua distribuzione per classe: 7 delle 9 applicazioni testate vengono classificate con recall pari o superiore all'83%, mentre una sola coppia di applicazioni (Canva e UPS) risulta sistematicamente confusa per una reale sovrapposizione delle caratteristiche di traffico. Questo conferma che, **nella maggioranza dei casi analizzati, è possibile distinguere l'applicazione specifica scaricata osservando esclusivamente il traffico di rete cifrato.**

Distinzione tra applicazioni diverse validato con risultato solido: 83.3% di accuratezza su 9 classi, errori concentrati e spiegati (Canva/UPS).

23. Fase 19 — Raccolta dati: Esecuzione dell'esperimento browsing / download / aggiornamento

E' stata condotta la raccolta dati per la distinzione tra tipologie di attività (navigazione nello store, download completo, aggiornamento di un'applicazione già installata).

23.1 Raccolta dati

Sono stati creati due nuovi script sul Raspberry Pi, con la stessa struttura del protocollo già validato per il download (test_cycle.sh): update_capture.sh (per la cattura di un aggiornamento reale) e browsing_capture.sh (per la cattura di navigazione libera dell'App Store, senza avviare alcun download).

update_capture.sh

```
#!/bin/sh
if [ -z "$1" ]; then
    echo "Uso: update_capture.sh <nome_app>"
    exit 1
fi

APP=$1
TS=$(date +%Y%m%d_%H%M%S)
PCAP="/tmp/cattura_update_${APP}_${TS}.pcap"
LOG="/tmp/log_update_${APP}_${TS}.txt"

echo "=== Assicurarsi che il blocco OAF sia disattivato ==="
uci delete appfilter.rule.app_list 2>/dev/null
uci commit appfilter
/usr/bin/oaf_rule reload

echo ""
echo ">>> Avvio la cattura tcpdump in background <<<"
tcpdump -i br-lan -w "$PCAP" &
TCPDUMP_PID=$!
sleep 1
echo "Cattura avviata (PID $TCPDUMP_PID) -> $PCAP"

echo ""
echo ">>> Avviare l'AGGIORNAMENTO dell'app (non una nuova installazione) <<<"
echo ">>> Quando l'aggiornamento è COMPLETATO, premere INVIO per fermare la cattura <<<"
read _

echo "=== Registro l'evento di riconoscimento ==="
ubus call appfilter dev_list > "$LOG"
cat "$LOG"

kill $TCPDUMP_PID

echo ""
echo "==== Ciclo completato ==="
echo "Log riconoscimento: $LOG"
echo "Cattura traffico: $PCAP"
```

browsing_capture.sh

```
#!/bin/sh
if [ -z "$1" ]; then
    echo "Uso: browsing_capture.sh <nome_app>"
    exit 1
fi

APP=$1
TS=$(date +%Y%m%d_%H%M%S)
PCAP="/tmp/cattura_browsing_${APP}_${TS}.pcap"
LOG="/tmp/log_browsing_${APP}_${TS}.txt"

echo "=== Assicurarsi che il blocco OAF sia disattivato ==="
uci delete appfilter.rule.app_list 2>/dev/null
uci commit appfilter
/usr/bin/oaf_rule reload

echo ""
echo ">>> Avvio la cattura tcpdump in background <<<"
tcpdump -i br-lan -w "$PCAP" &
TCPDUMP_PID=$!
sleep 1
echo "Cattura avviata (PID $TCPDUMP_PID) -> $PCAP"

echo ""
echo ">>> Aprire l'App Store, cercare app, scorrere le pagina, guardare screenshot <<<"
echo ">>> NON scaricare/aggiornare nulla, solo navigare per circa 1 minuto <<<"
echo ">>> Premere INVIO qui per fermare la cattura <<<"
read _

echo "=== Registro l'evento di riconoscimento ==="
ubus call appfilter dev_list > "$LOG"
cat "$LOG"

kill $TCPDUMP_PID

echo ""
echo "=== Ciclo completato ==="
echo "Log riconoscimento: $LOG"
echo "Cattura traffico: $PCAP"
```

Aggiornamento: si è verificato che le applicazioni già disinstallate durante i test precedenti non avessero più aggiornamenti pendenti (essendo state rimosse). Sono state quindi individuate 8 applicazioni con un aggiornamento reale disponibile al momento della raccolta (Vinted, Ryanair, Reddit, Maps, Teams, Twitch, Instagram, LinkedIn), catturando un campione ciascuna, un aggiornamento non è ripetibile per la stessa versione applicativa, per cui non è stato possibile ottenere ripetizioni della stessa applicazione per questa categoria, a differenza delle altre due.

Browsing: un primo tentativo di cattura ha prodotto un file di soli 1.5 KB, praticamente privo di traffico utile, diagnosticato come dovuto a un tempismo troppo rapido tra l'avvio della navigazione e la pressione del tasto Invio per fermare la cattura, verificato osservando in tempo reale la crescita del file con un ciclo di controllo (**while true; do ls -la ...; sleep 2; done**, in sostituzione del comando watch non disponibile su questa immagine BusyBox). Il protocollo è stato quindi standardizzato su una navigazione attiva di circa un minuto (classifiche, sezioni in evidenza, filtri), risultando in catture nell'ordine di 53-97 MB, un range compatto e ragionevolmente coerente tra le 9 ripetizioni raccolte.

Dataset raccolto: Download 54 campioni (9 app, riutilizzate per la classificazione delle app), Browsing 9 campioni, Aggiornamento 8 campioni (applicazioni diverse, un campione ciascuna).

23.2 Individuazione e correzione di un artefatto nei dati

Un primo risultato (accuratezza 87.7% leave-one-out) è stato messo in dubbio osservando che la feature `n_pause_lunghe` non era mai stata calcolata per le 54 catture di download (introdotta solo in questo esperimento), risultando quindi assente e sostituita da un valore zero nell'unione dei dataframe. Questo introduceva un artefatto: il modello poteva aver imparato a riconoscere la categoria Download semplicemente dall'assenza sistematica di pause lunghe (zero per costruzione), anziché da un comportamento di traffico realmente osservato.

La feature è stata ricalcolata per tutte le 54 catture di download, applicando la stessa funzione di estrazione ai file .pcap originali già disponibili. Dopo la correzione, l'accuratezza è scesa a 82.2%, con un crollo marcato del recall sulla categoria Aggiornamento (da 50% a 12%), a conferma che il risultato iniziale era in parte sovrastimato dall'artefatto.

Episodio rilevante per la tesi: un errore metodologico individuato e corretto autonomamente, con impatto misurato (87.7% -> 82.2%). Il risultato corretto (82.2%) è quello di riferimento.

23.3 Risultato principale e verifica di robustezza

Il risultato finale, sul dataset con la feature corretta, è un'accuratezza dell'82.2% (leave-one-out) nella distinzione tra le tre categorie, contro un livello atteso del 33.3% per una scelta casuale su 3 classi. La verifica di robustezza (split train/test indipendenti, stratificati) conferma il risultato con una media dell'80.5% (deviazione standard 7.1%, range 63.6%-86.4%).

23.4 Approfondimenti sul risultato

Per interpretare meglio il margine di errore residuo, sono stati condotti due approfondimenti mirati (non nel tentativo di sostituire il risultato principale, ma per comprenderne meglio la struttura):

Configurazione	Accuratezza	Interpretazione
3 classi, dataset completo (risultato principale)	82.2%	Riferimento
Solo Download vs Browsing (2 classi)	93.8%	L'errore residuo è concentrato quasi interamente nella categoria Aggiornamento
3 classi, dataset bilanciato per numerosità	75.0%	Test più severo: parte del risultato principale beneficia dello sbilanciamento verso Download (56 campioni contro 8-9)

La categoria Aggiornamento resta la più difficile da riconoscere in ogni configurazione testata (recall 12-38% a seconda della configurazione), coerentemente con la sua base campionaria strutturalmente più limitata (8 applicazioni diverse, non ripetibili per la natura dell'evento) e più eterogenea rispetto alle altre due categorie.

Distinzione tra tipologie di attività completato: risultato principale 82.2%, verificato e onestamente interpretato, con limiti chiaramente dichiarati

Idea per l'Enforcement dimostrativo

Coerentemente con la proposta di tesi, che specifica esplicitamente come l'obiettivo dell'enforcement non sia disabilitare l'applicazione sul dispositivo mobile, ma dimostrare l'integrazione tra riconoscimento e controllo a livello di rete, è stata proposta la seguente pipeline dimostrativa:

- Un dispositivo di test scarica un'applicazione (traffico catturato normalmente)
- Il classificatore sviluppato analizza la cattura e riconosce quale applicazione è stata scaricata
- Se l'applicazione riconosciuta rientra in una lista di interesse (policy definita), viene applicata automaticamente la regola OAF corrispondente (uci set `appfilter.rule.app_list` + attivazione di `tcp_rst`, riutilizzando gli script `oaf_block.sh/oaf_free.sh` già predisposti), così che i tentativi successivi del medesimo dispositivo verso quel servizio vengano bloccati

- L'azione intrapresa viene registrata in un log (applicazione riconosciuta, timestamp, regola applicata), a scopo dimostrativo

Nota metodologica: poiché la classificazione accurata richiede l'osservazione dell'intera sessione di traffico (la classificazione precoce sui soli pacchetti iniziali non ha dato risultati affidabili), l'enforcement proposto agisce necessariamente sui flussi successivi al download classificato, non durante il download stesso, come limite/caratteristica del sistema.